



AI TECH GTM · REFERENCE WHITEPAPER

Foundation Model Security for the Enterprise

A defence-in-depth reference for regulated organisations
deploying frontier and open foundation models

Author Sanchit Alekh · **Published** 2026-08-27 · **Classification** Google Cloud | Proprietary & Confidential

Contents

01

Executive summary

02

Why foundation-model security is structurally different

03

What the frontier labs actually do — and what transfers

04

Securing model artefacts — the RAND ladder and what enterprises should target

05

The agentic attack surface

06

Prompt injection: an honest reality check

07

A defence-in-depth reference architecture

08

Evaluation, red teaming and assurance

09

Standards, regulation and audit evidence

10

A 90-day adoption path

11

References

01 Executive summary

Foundation models change the shape of enterprise attack surface. A traditional application accepts data on well-defined channels and executes code the vendor wrote and the security team reviewed. A model-driven application blends control-plane instructions and untrusted data into a single probabilistic reasoning step, then hands that reasoning tools that can act on production systems. The classical separation between what the system was told to do and what it was told to work on collapses inside the model. Every established appsec assumption — deterministic control flow, provable input validation, allow-listed sinks — weakens the moment the reasoning core becomes a language model.

Three developments in 2025 reshaped the threat model. First, the frontier labs' safety frameworks converged on capability-triggered controls: Anthropic activated its Responsible Scaling Policy AI Safety Level 3 (ASL-3) for Claude Opus 4 in May 2025, OpenAI's Preparedness Framework v2 (15 April 2025) formalised "High" and "Critical" capability thresholds for biological/chemical, cybersecurity and self-improvement risks, and Google DeepMind's Frontier Safety Framework v3 (22 September 2025) added a Critical Capability Level for harmful manipulation and expanded review of misalignment and shutdown resistance. Second, the Model Context Protocol (MCP) put a fast-moving, largely untrusted supply chain of tools in front of enterprise agents, culminating in CVE-2025-6514 (CVSS 9.6, disclosed 9 July 2025) — a remote code execution flaw in `mcp-remote` that gave any hostile MCP server a path to complete client compromise. Third, the EU AI Act's general-purpose AI (GPAI) obligations became applicable on 2 August 2025, with the bulk of enforcement provisions and systemic-risk penalties for GPAI providers taking effect on 2 August 2026.

The consequence for a CISO is uncomfortable but clear: **indirect prompt injection has no complete defence today**, and no responsible security programme can premise itself on eliminating it. The task is instead to reduce the probability of exploitation, contain blast radius when it occurs, and produce auditable evidence that the residual risk is understood and priced. This whitepaper argues that enterprises should adopt a tiered control model (T1–T4) matched to the sensitivity of what an agent can read and the reversibility of what it can do; align each tier to concrete Google Cloud controls (Model Armor, Security Command Center AI Protection, Sensitive Data Protection, VPC Service Controls, CMEK/Cloud HSM, Vertex AI Agent Engine); and instrument the whole system with continuous evaluation and red teaming rather than one-off pre-deployment tests.

The single most important claim in this document

Indirect prompt injection is an unsolved problem. Every published defence — Constitutional Classifiers, instruction hierarchy, spotlighting, guardrail models, CaMeL — reduces the attack success rate; none drives it to zero on adaptive attackers. Any architecture that assumes otherwise is unsafe.

38

attack vectors on model weights catalogued by RAND (RR-A2849-1, May 2024)

10

risks in OWASP Top 10 for LLM Applications, 2025 edition

CVSS 9.6

severity of CVE-2025-6514 in `mcp-remote` (Jul 2025)

The remainder of this document sets out: why classical appsec falls short (§2); how frontier labs gate capability and what transfers to a non-training enterprise (§3); realistic model-weight protection targets against RAND SL1–SL5 (§4); the agentic attack surface and its kill chain (§5); an honest accounting of what prompt-injection defences buy (§6); a defence-in-depth reference architecture and its Google Cloud realisation (§7); an evaluation and red-team programme (§8); the standards and regulatory evidence expected by auditors (§9); and a sequenced 90-day adoption path (§10).

02 Why foundation-model security is structurally different

Enterprise security programmes have four decades of accumulated practice on top of a small number of load-bearing assumptions: code is deterministic; inputs traverse typed interfaces; trust boundaries are enforced by process, memory, or network controls; and privilege can be enumerated and audited. Foundation models weaken every one of these assumptions simultaneously. The consequence is not that existing controls are useless — they remain necessary — but that they are insufficient, and the gaps sit precisely where the highest-impact failures now cluster.

2.1 Instruction/data collapse

In a conventional web application, the SQL string and the user-supplied `WHERE` value live in different variables and are separated at execution time by a parameterised query. In an LLM pipeline, the system prompt, the developer-authored instructions, the retrieved document, the tool response and the end-user message are all concatenated into a single token stream and processed by the same next-token predictor. The model has no reliable, cryptographically enforced way to distinguish "what the operator wants" from "what the untrusted content says". Every mitigation for this problem — role tags, delimiters, spotlighting, instruction hierarchy — is a *soft* convention that the model has been trained to respect, not a hard boundary the runtime enforces. Adaptive attackers routinely overcome soft conventions.

2.2 Non-determinism and probabilistic reasoning

A traditional control (e.g. an ACL) evaluates the same input to the same outcome every time. A model-driven control evaluates the same input to a distribution over outcomes. Even at temperature zero, model versions change under the enterprise's feet, retrieval contexts vary, and small perturbations in phrasing shift refusal behaviour. This has three concrete consequences: (a) regression is continuous, not episodic — a benign prompt that was refused yesterday may be answered today; (b) point-in-time penetration tests decay in value quickly and must be replaced by continuous evaluation; and (c) safety and security metrics need confidence intervals, not single-point pass/fail assertions.

2.3 Opaque provenance

A traditional binary can be attested with a supply-chain signature that traces to source code, a build system and an identified maintainer. A foundation model's weights are an opaque tensor whose behaviour is a function of a training corpus that is typically neither disclosed in full nor practically auditable. Fine-tuning and adapter layers compound the problem: a LoRA rank-16 adapter is a few dozen megabytes that can implant a backdoor, be distributed on a model hub, and never appear on any software bill of materials. Model-supply-chain attestation — Sigstore for artefacts, MLflow-style lineage for training runs, and hash-pinned model registries — is deployable today but nowhere near the maturity of software supply-chain tooling.

2.4 Emergent capability

Frontier models acquire capabilities discontinuously as scale and post-training regimes evolve. A model that could not write exploit code six months ago may do so competently today; a model that refused a class of query at launch may comply after a routine post-training update. Static risk assessments produced at procurement time have short half-lives. This is the core reason the frontier labs adopted capability-triggered policies (see §3): the safety envelope is a moving target that must be re-measured, not asserted once.

2.5 Agentic action closes the loop

When a model merely produces text, the blast radius of a mis-generation is bounded by what a human does with that text. When the model is given tools — a database connector, an email API, a payments endpoint, an MCP server pool — the blast radius is bounded only by the union of privileges granted to the agent. Every classical privilege-escalation pattern (confused deputy, over-broad service accounts, path traversal) is available to an attacker who can steer the model. New patterns emerge on top: memory poisoning, tool poisoning, cross-server tool shadowing, "excessive agency" as codified in OWASP LLM06:2025.

The load-bearing insight

Foundation-model security is not "web app security plus a chatbot". It is a new class of adversarially-influenced automation where the control plane and the data plane share a single probabilistic execution engine. Controls must be redesigned around that fact, not bolted onto assumptions that no longer hold.

Assumption of classical appsec	Status under foundation models	Practical consequence
Code is deterministic	False (probabilistic reasoning)	Regression is continuous; evals must run in CI, not at gate.
Instructions and data are separable	False (single token stream)	Indirect prompt injection is a first-class risk on every input.
Provenance can be attested	Weak (opaque weights, adapters)	Model registry must hash-pin; adapters need explicit review.
Capability is stable across versions	False (emergent, post-training drift)	Deployment gates must be re-run on every model or adapter change.
Privilege can be enumerated statically	Partial (dynamic tool selection)	Least privilege must be enforced at the tool broker, not the code.

03 What the frontier labs actually do — and what transfers

The three organisations that train the most capable general-purpose models have all published capability-triggered safety and security frameworks. They differ in vocabulary, in the exact capability areas they emphasise, and in the assurance rigour they claim. They converge on the same operating pattern: define a small set of dangerous capability thresholds; evaluate each new model against them; and require a specific set of deployment and security mitigations before, and continuously after, the threshold is crossed. This section summarises the three frameworks as of Q3 2025, then extracts what an enterprise that does *not* train frontier models should nevertheless carry over.

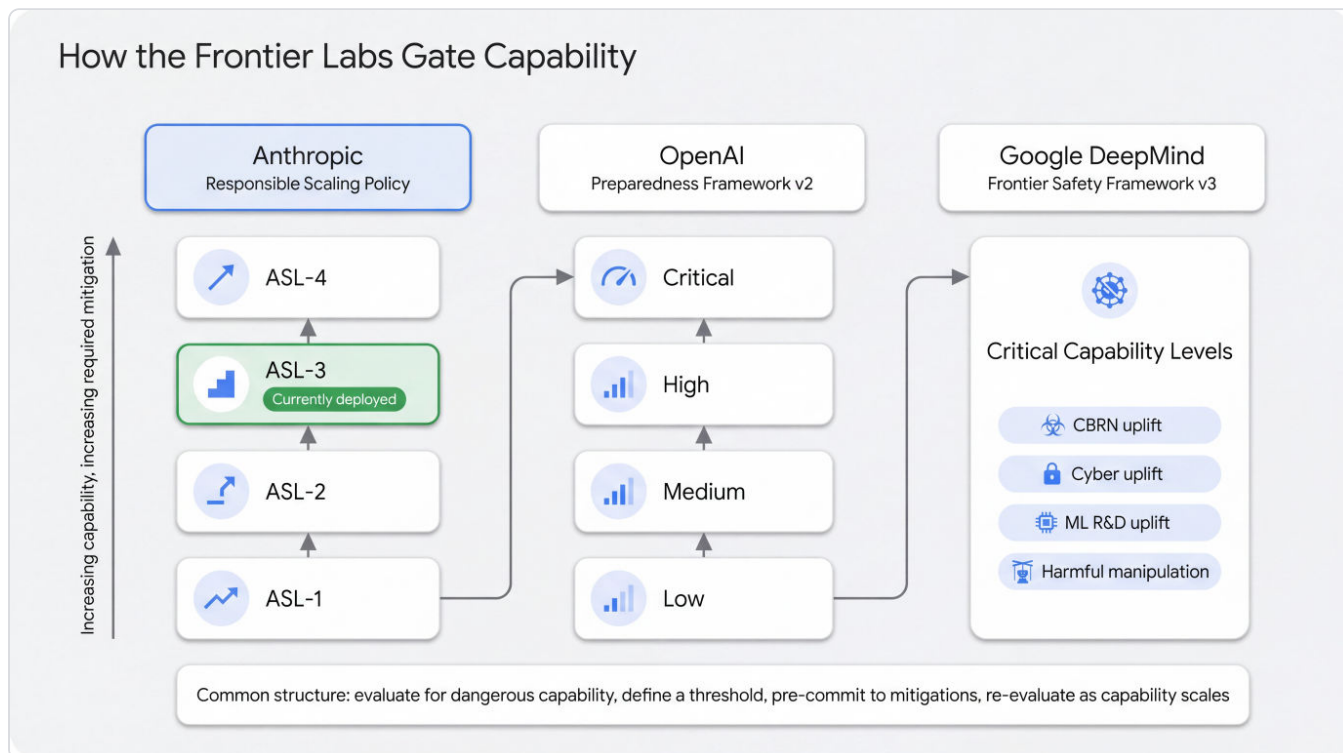


Figure 1. The three major frontier-lab frameworks share an evaluate-then-gate control pattern; they disagree on vocabulary and on which capability domains dominate the risk envelope.

3.1 Anthropic — Responsible Scaling Policy (RSP)

Anthropic's RSP defines AI Safety Levels (ASL) that ratchet up in response to demonstrated model capability. **ASL-2** is the baseline standard applied to production Claude models. **ASL-3** adds deployment measures aimed at jailbreak robustness (particularly for uplift on chemical, biological, radiological and nuclear — CBRN — misuse) and a security standard scoped to non-state adversaries: hackers, organised crime, terrorist organisations, corporate espionage, and unsophisticated insiders. Sophisticated insiders and nation-state APT-level actors fall to the future **ASL-4** standard.

In May 2025 Anthropic activated ASL-3 protections for Claude Opus 4 — the first industry deployment under an ASL-3 regime. The decision was framed as precautionary: Anthropic stated it could no longer confidently make a "risk is low" case, not that it had proven the risk was high. The RSP has since been iterated (v2.2 was published in October 2025 with more qualitative descriptions of thresholds and mitigation objectives).

3.2 OpenAI — Preparedness Framework v2

OpenAI's Preparedness Framework v2 (15 April 2025) tracks three capability categories where sufficient capability is judged to enable severe, irreversible harm: **Biological and Chemical**, **Cybersecurity**, and **AI Self-improvement**. Additional **Research Categories** are studied but do not yet trigger mandatory safeguards. Each tracked capability has two thresholds — **High** and **Critical** — and each threshold binds a specific set of pre-deployment and continuous safeguards. v2 explicitly positions safety as an operating discipline (monitoring, security engineering, alignment safeguards, formal oversight) rather than a pre-launch checkpoint.

3.3 Google DeepMind — Frontier Safety Framework v3

The DeepMind FSF was refreshed on 22 September 2025 to version 3. It is organised around **Critical Capability Levels (CCLs)** in three families: **misuse CCLs** (CBRN uplift, offensive cyber, and — new in v3 — **harmful manipulation at scale**), **machine-learning R&D CCLs**, and **misalignment CCLs** covering scenarios where models resist modification or shutdown. v3 requires **safety case reviews** not only prior to external launch but also for large-scale internal deployment once a CCL threshold is reached. In practice a safety case is a written argument, supported by evidence, that residual risk at that CCL is acceptable given specified mitigations.

3.4 Convergence and the ecosystem

The three frameworks converge on a small number of design choices: (a) capability, not intent, triggers controls; (b) a small number of high-severity thresholds bind concrete mitigations; (c) continuous evaluation and red teaming are required in production, not just at launch; and (d) organisational oversight sits outside the product team. The **Frontier Model Forum** provides the industry-level venue for aligning these practices. Public-sector counterparts include the **UK AI Security Institute** (renamed from AI Safety Institute in February 2025, signalling a narrowed remit on national-security-relevant capabilities) and the **US Center for AI Standards and Innovation (CAISI)**, which succeeded the US AISI within NIST.

Framework	Owner	Version / date	Capability vocabulary	Notable 2025 event
Responsible Scaling Policy	Anthropic	v2.2, Oct 2025	ASL-2, ASL-3, ASL-4	ASL-3 activated for Claude Opus 4 (May 2025)
Preparedness Framework	OpenAI	v2, 15 Apr 2025	High / Critical, three Tracked Categories	Continuous-monitoring model formalised
Frontier Safety Framework	Google DeepMind	v3, 22 Sep 2025	Critical Capability Levels (misuse, ML R&D, misalignment)	Harmful-manipulation CCL added; internal-deployment safety cases required

3.5 What transfers to an enterprise that does not train models

Very few enterprises will ever train a frontier model. Nevertheless, four practices from the frontier-lab playbook transfer directly to a large regulated organisation:

- 1 Capability-triggered, not vendor-triggered, controls.** The trigger for tighter controls should be the capability granted to a deployed agent (tool set, data access, reversibility), not the identity of the model vendor. A T4 agent (see §7) with production write access warrants ASL-3-style deployment discipline regardless of which model powers it.
- 2 Written safety cases for high-impact agents.** For any agent that can move money, change customer records, deploy code, or communicate externally on behalf of the enterprise, require an evidence-backed safety case before launch and on every material change. This is a light-weight adaptation of DeepMind's v3 requirement.

- 3 **Continuous, not point-in-time, evaluation.** Move golden-task and red-team evaluations into CI (see §8). Frontier labs run automated evaluations continuously; enterprises must too because their agents change more often than the underlying models do.
- 4 **Independent oversight.** A named individual, outside the product engineering line, must be accountable for the go/no-go decision on capability-critical agents. In practice this is a role for a Head of AI Assurance or the CISO's chief architect.

04 Securing model artefacts — the RAND ladder and what enterprises should target

Model weights are high-value assets in two distinct senses: they are expensive to reproduce (frontier training runs cost hundreds of millions of dollars), and once stolen they can be deployed by an adversary without any of the safety mitigations the original developer built. The RAND Corporation's report *Securing AI Model Weights* (RR-A2849-1, Sella Nevo et al., May 2024) is the reference work on the topic. It catalogues **38 attack vectors** across nine categories (running from running-model access and network breach through supply-chain compromise and physical/insider access), models attacker capacity from opportunistic criminals up to top-tier nation-state programmes, and proposes **five security levels (SL1–SL5)** with benchmark control sets sized to the threat.



Figure 2. RAND's SL1–SL5 ladder maps attacker capacity to a benchmark control set; enterprises that consume frontier models via API inherit the vendor's level for the base model but own the level for every fine-tune and adapter they hold.

4.1 The ladder in enterprise terms

Level	Adversary the level should resist	Typical benchmark controls (illustrative)	Realistic enterprise target
SL1	Amateur/opportunistic attackers	Basic access controls, patched OS, MFA on admin surfaces	Any weights of business value
SL2	Professional cybercriminals, moderate insider	Hardware-backed key management, network segmentation, comprehensive logging	Fine-tunes of proprietary data
SL3	Well-resourced criminal groups, small nation-states	Confidential computing, air-gapped signing, formal insider-threat programme	Weights whose leakage would breach regulation or IP

Level	Adversary the level should resist	Typical benchmark controls (illustrative)	Realistic enterprise target
SL4	Standard nation-state programmes	Two-party controls on weight access, egress bandwidth caps on weight-hosting environments, red-team validated	Only for organisations training frontier or near-frontier models
SL5	Top-tier nation-state cyber programmes	Full weight-storage isolation, formal-methods-verified control paths, continuous adversary emulation	Practically unattainable outside a handful of frontier labs and state actors

The great majority of enterprises that consume foundation models via managed APIs (Vertex AI, Anthropic API, Azure OpenAI) inherit the *vendor's* posture for the *base* model. What they own — and what most under-invest in — is the security of everything they build on top: fine-tunes, adapters, embeddings, and the retrieval corpora that carry proprietary data into inference.

4.2 Where enterprises actually sit

A pragmatic distribution for regulated European enterprises in 2026:

- **Base foundation models consumed via managed API:** vendor-provided, typically SL2–SL3 for major cloud providers.
- **Enterprise fine-tunes and LoRA adapters holding proprietary data:** should target **SL2 minimum, SL3 for regulated data** (payment, health, insider information). This is the target most organisations should first close the gap to.
- **Self-hosted open-weight models handling regulated data:** SL3. This is more expensive than most teams estimate, because it requires hardware-backed key management (Cloud HSM or equivalent), CMEK on all model storage, VPC-SC-style perimeter enforcement on inference endpoints, and comprehensive audit logging.
- **Own-trained frontier models:** SL4 is the appropriate aspiration; SL5 is a research-grade objective that few organisations can credibly claim.

4.3 Supply chain: adapters and third-party models

The 2025 wave of model-supply-chain incidents established the pattern: a benign-looking adapter or fine-tune on a public model hub can encode a backdoor that activates on a trigger phrase, and the sheer volume of published derivatives makes manual review infeasible. The controls that work:

- **Hash-pin every model artefact.** No `latest` tag, no floating adapter reference. Every registry entry pins to a content hash and a signed provenance record.
- **Attest artefacts.** Sigstore or an equivalent transparency-log-backed signature for every model uploaded to the internal registry. In-toto attestations for every training or fine-tuning run, capturing dataset hashes, code commit, and executing identity.
- **Behavioural admission testing.** No model — base, fine-tune or adapter — enters the internal registry until it has passed a fixed golden-task suite (see §8) including trigger-word probes designed to surface backdoors.
- **Egress control on model storage.** Weights live in a CMEK-encrypted, VPC-SC-perimeter-protected bucket with egress logging and, at higher tiers, egress bandwidth caps borrowed from the RAND SL4 pattern.

Cheapest high-value control

If only one supply-chain control ships this quarter, make it hash-pinning with mandatory signature verification in the model registry. It is inexpensive, breaks a very large fraction of realistic supply-chain attacks, and provides audit evidence usable against ISO/IEC 42001 and the EU AI Act GPAI transparency obligations.

4.4 Fine-tune and adapter risk

Fine-tuning is not a neutral customisation step. Two categories of failure recur:

- 1 **Safety regression.** Instruction-tuned refusal behaviour is often the first thing degraded by domain fine-tuning. A model that refused to draft a phishing email at base checkpoint may comply after a fine-tune focused on marketing copy. Every fine-tune must re-run the safety eval suite; a regression above a fixed threshold blocks promotion.
- 2 **Data leakage through the tuned weights.** Fine-tuning on sensitive corpora embeds fragments of those corpora into the weights; extraction attacks against fine-tuned models are an active research area. Where regulation requires data-subject deletion, this is a legal problem, not just a security one. The mitigations available today are (a) differential-privacy-aware training regimes, (b) tokeniser-level PII redaction on the fine-tuning corpus, and (c) architectural preference for RAG (data outside the weights, in a governed store) over fine-tuning wherever the use case allows.

05 The agentic attack surface

An agent is a foundation model given (a) a persistent context or memory, (b) a set of tools it may call, and (c) an outer loop that lets it call those tools iteratively until a goal is judged achieved. Every one of those three additions expands the attack surface, and the interactions between them create attack classes that were not present in either classical software or in single-turn LLM applications.

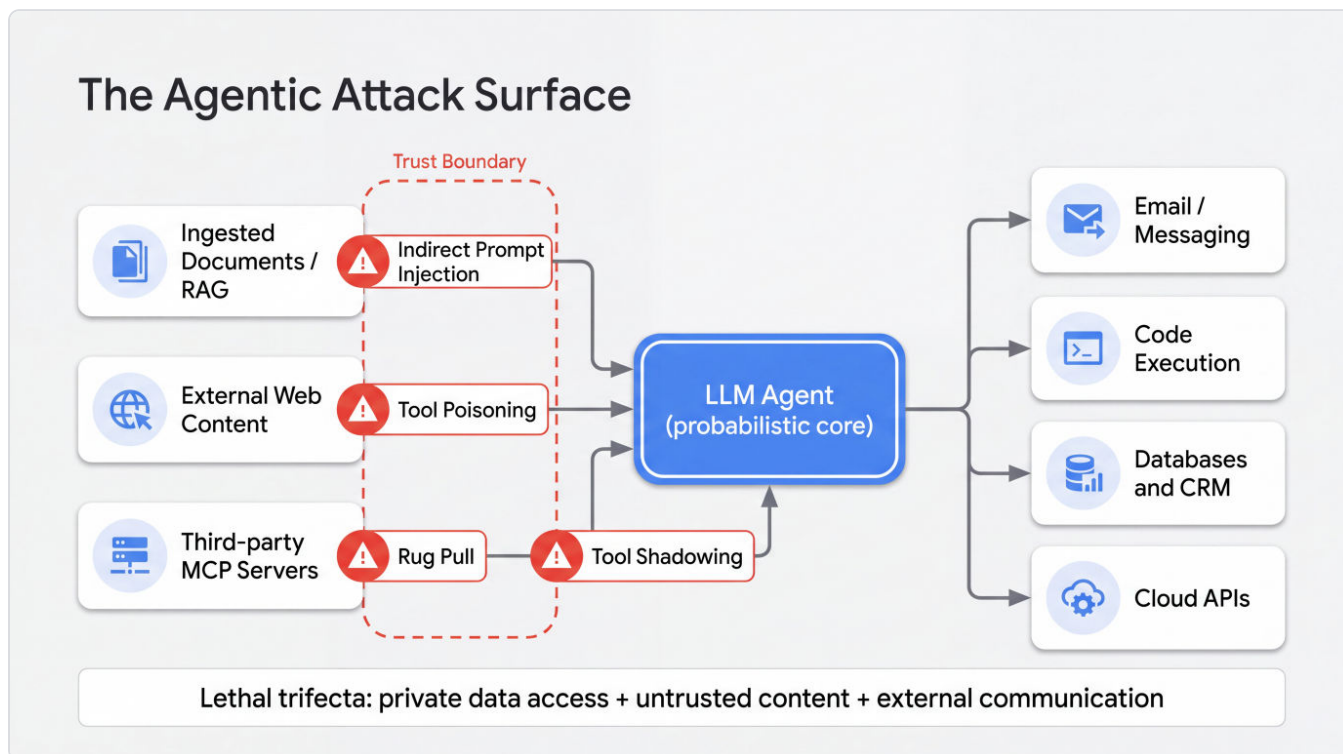


Figure 3. The agentic attack surface. Untrusted content reaches a probabilistic reasoning core that holds privileged tools; every arrow that enters the core is a candidate injection channel and every arrow that leaves it is a candidate exfiltration channel.

5.1 A taxonomy that maps to OWASP

The OWASP GenAI Security Project's *Top 10 for LLM Applications (2025)* is the shared vocabulary that security architects, auditors and product teams should adopt. The full list:

ID	Risk	Where it bites hardest in agents
LLM01	Prompt Injection	Retrieved documents, tool outputs, MCP responses
LLM02	Sensitive Information Disclosure	System prompt leakage, PII in outputs, embedding inversion
LLM03	Supply Chain	Model hubs, MCP servers, third-party adapters
LLM04	Data and Model Poisoning	Fine-tuning corpora, RAG index, agent memory
LLM05	Improper Output Handling	Direct rendering, eval, unsafe SQL synthesis
LLM06	Excessive Agency	Over-broad tool sets, unbounded autonomy, silent auto-execute
LLM07	System Prompt Leakage	Introspection prompts, error-message oracles
LLM08	Vector and Embedding Weaknesses	Poisoned corpora, embedding-space adversaries, cross-tenant retrieval

ID	Risk	Where it bites hardest in agents
LLM09	Misinformation	Confident hallucination in high-stakes workflows
LLM10	Unbounded Consumption	Cost/DoS via tool loops, expensive tool invocation

OWASP's newer **Top 10 for Agentic Applications** (announced in December 2025) sharpens several of these to agent-specific patterns — notably **Memory Poisoning**, **Tool Misuse**, **Agent Behaviour Hijacking**, and **Identity and Privilege Abuse**. Enterprises should carry both lists; the LLM Top 10 aligns to appsec vocabulary, the Agentic Top 10 aligns to how agents actually fail in production.

5.2 The indirect prompt injection kill chain

The dominant new attack pattern is indirect prompt injection. The attacker does not talk to the model; the attacker plants a payload in content the model will later ingest. The chain typically runs in five stages.

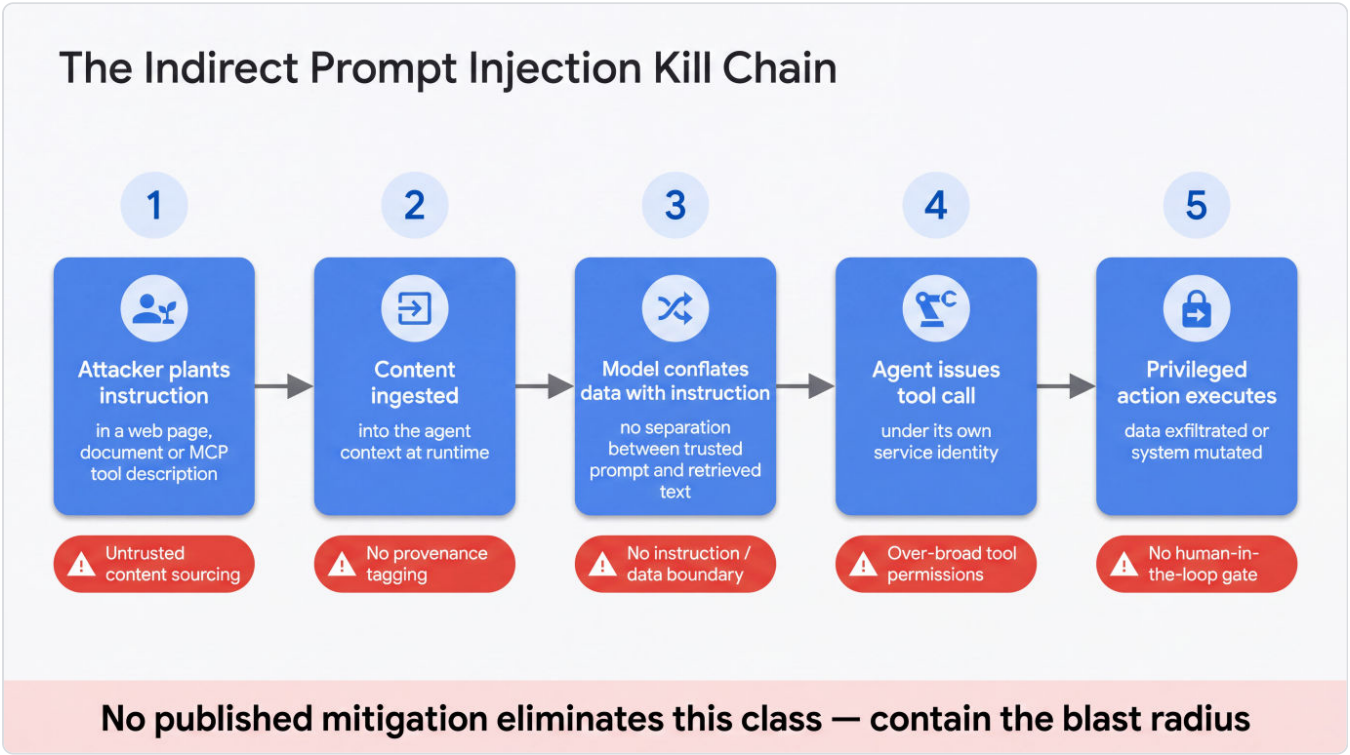


Figure 4. The five-stage indirect prompt injection kill chain: plant, retrieve, execute, exfiltrate, persist — with a defensive control candidate at each transition.

- 1 **Plant.** The attacker writes malicious instructions into a document, a webpage, a GitHub issue, a support ticket, an email footer, a calendar invite description, or an MCP tool response.
- 2 **Retrieve.** The victim agent ingests that content via RAG, browsing, email reading, calendar summarisation, or a tool call.
- 3 **Execute.** The reasoning core treats the injected text as instructions from the operator. It selects tools and constructs arguments accordingly.
- 4 **Exfiltrate or act.** The agent performs the attacker's chosen action: emailing sensitive data to an attacker-controlled address, sending a rendered image whose URL encodes stolen data, calling a write-capable tool with attacker-supplied parameters, or overwriting its own memory to embed a durable backdoor.
- 5 **Persist.** The compromise survives the immediate task: memory-poisoning attacks add attacker-controlled facts to long-term storage; tool-preference manipulation biases future tool selection; a MCP "rug pull" waits until the operator has approved the server, then swaps in a hostile tool definition on a subsequent connection.

Simon Willison's **lethal trifecta** formulation captures the necessary and (approximately) sufficient conditions for an indirect-injection attack to cause enterprise-scale harm: an agent that combines **access to private data**, **exposure to untrusted content**, and **the ability to communicate externally** is at risk regardless of how carefully its prompts are engineered. If any one of the three is absent, the harm surface collapses dramatically. This is the single most useful architectural test to apply to a proposed agent design.

The lethal-trifecta test

For every proposed agent, ask three questions. Does it read private or regulated data? Does it ingest content from any source that is not fully trusted? Can it communicate externally (email, HTTP, tool calls with side effects, rendering URLs)? If the answer is "yes" to all three, containment must carry the security case — reliance on the model's own resistance to injection is inadequate.

5.3 MCP-specific risks

The Model Context Protocol was designed to standardise how agents discover and invoke external tools. Its rapid adoption during 2025 created a large, largely un-curated tool ecosystem. Enterprises adopting MCP need to defend against a specific attack family:

- **Tool poisoning.** A hostile MCP server returns tool definitions whose descriptions contain injection payloads. The agent reads the tool description as part of its reasoning context, and the payload steers subsequent tool selection.
- **Rug pulls.** A benign-looking MCP server changes its tool definitions after initial approval. Because approvals are frequently persistent, later tool sets never re-enter human review.
- **Cross-server tool shadowing.** Multiple MCP servers connected to the same agent expose tools with overlapping names or descriptions; the more persuasive description wins the agent's selection, regardless of which is authorised.
- **Output injection.** Tool responses carry injection payloads; because the agent already trusts its own tools, output content is often processed with less scepticism than model input from users.
- **Client-side RCE. CVE-2025-6514** (CVSS 9.6, JFrog Security Research, 9 July 2025) demonstrated that `mcp-remote` versions 0.0.5 through 0.1.15 could be induced to execute arbitrary OS commands when connected to a hostile MCP server, via crafted `authorization_endpoint` URL response handling. The fix is version 0.1.16 or later; the general lesson is that MCP client code is now a first-class attack surface and must be patched with the same discipline as browser or SSH clients.

5.4 Agent-to-Agent (A2A) and multi-agent risks

Google's Agent-to-Agent (A2A) protocol and equivalent multi-agent orchestration patterns extend the trust boundary further. Every peer agent becomes a source of both instructions and untrusted content; every delegation becomes a place where the caller's authority may be exercised by the callee, exactly the classical **confused-deputy** pattern first described by Norm Hardy in 1988. In the AI agent case the deputy holds credentials granted to the *builder* or *admin* who deployed the agent, but takes actions on behalf of *any user* (or peer agent) who can invoke it. Downstream systems observe the admin's identity, not the caller's. IAM is bypassed by design unless propagated caller identity is enforced.

The controls that work for A2A are the same as for MCP: mutual authentication of peer agents; explicit propagation of the invoking principal into every downstream call; policy at the tool broker layer that considers *both* the calling agent's identity and the ultimate human principal; and audit records that reconstruct the full authority chain rather than only the immediate call.

5.5 Memory persistence attacks

Long-lived agents store context across sessions. Memory becomes an attack surface in three ways:

- 1 **Poisoning.** An earlier task writes attacker-controlled "facts" into memory; a later, unrelated task reads them and treats them as trusted.
- 2 **Exfiltration through memory.** Memory that spans users or tenants can leak information from one caller to another if isolation is not strictly enforced at the memory-store layer.
- 3 **Compounding drift.** Even without malice, memory that accumulates hallucinations or user-corrected errors that were never actually applied can drive an agent's reasoning progressively off-policy.

Mitigations centre on strict per-caller (or per-tenant) memory isolation, provenance stamps on every memory write (who wrote it, from what source, at what time), TTLs on all memory records absent an explicit business reason to retain them, and read-time trust levels that keep memory-derived content from being treated with the same weight as system-prompt content.

06 Prompt injection: an honest reality check

The frontier labs, academic groups and defensive-tooling vendors have published a rich literature of prompt-injection defences. Each reduces attack success on specific attack distributions. None solves the problem. This section characterises what each family of defence actually buys, so that architects can price residual risk correctly rather than assume it away.

6.1 The defences, briefly

Defence family	Representative work	What it buys	What it does not buy
Instruction hierarchy	OpenAI, Wallace et al. (2024)	Preference for system > developer > user instructions; measurable reduction in overriding on trained distributions	Cannot cryptographically enforce hierarchy; adversarial content still wins on out-of-distribution attacks
Spotlighting / delimiting	Microsoft Research	Reduces model's tendency to treat data as instructions when data is marked with reserved tokens	Trivially defeated by attackers who include the reserved tokens in their payload
Constitutional Classifiers	Anthropic (2025)	Large measured reduction in successful jailbreaks on specific attack sets; deployed as part of Claude's Opus 4 ASL-3 posture	Adds latency and cost; residual attack surface remains; classifier itself becomes an attack target
Guardrail models	Broad industry (Model Armor, NeMo Guardrails, others)	Independent input/output screening for injection patterns, PII, malicious URLs, jailbreak signatures	Detection ceilings on novel attacks; false positives; increases blast radius when misconfigured
CaMeL — Capabilities for Machine Learning agents	Google DeepMind (2025)	Provable enforcement of security policies at the tool-call boundary; on AgentDojo, solves 77% of tasks with provable security vs 84% undefended	Requires re-architecting agents around a capability plane; not a drop-in defence
Dual-LLM pattern	Simon Willison and others	Structural separation of a <i>privileged</i> planner (never exposed to untrusted content) from a <i>quarantined</i> executor that sees untrusted content but holds no tools	Constrains what agents can do; loses some of the value of tool-using agents
System-prompt hardening	Widely practised	Marginal improvement on naive attacks; useful hygiene	Not a security control by itself; over-reliance is common

Two observations from the table. First, the strongest results in 2025 came from architectural approaches — **CaMeL** (capability-based enforcement at the tool boundary) and **dual-LLM** patterns (structural separation of privileged and untrusted execution paths) — not from making the model itself more robust. Second, even the strongest architectural approaches trade some agent utility for security, and the trade is not always acceptable to the business owner. That negotiation must be explicit.

Do not price prompt-injection defences as if they were input validation

Input validation for SQL injection is a hard boundary: once parameterised queries are correctly implemented, the class of attack is closed. Prompt-injection defences are probabilistic and adversarial. The correct mental model is a WAF against a motivated attacker: raises the bar, buys detection time, does not remove the need for downstream containment.

6.2 Why containment must carry the security case

Because no defence closes the class, the load-bearing control is what the agent is *allowed to do* if it is successfully steered by an attacker. Containment operates at three layers:

- 1 **Least-privilege tool sets.** Every agent gets the smallest set of tools sufficient for its stated purpose, and every tool call is authorised against the principal's own permissions — never the agent's admin identity.
- 2 **Egress control.** Every outbound side effect (email, HTTP, tool call with write semantics, URL rendering) traverses a policy point that can allow, deny, or step up to human confirmation.
- 3 **Reversibility.** For any action whose reversal is expensive or impossible (money movement, external communication, code deployment, customer-record change), a human-in-the-loop gate is the security control that carries the residual injection risk. Automation of such actions is a business decision that must be made with the full injection-risk picture on the table.

6.3 What is deployable today vs active research

Category	Deployable in production today	Active research / not yet enterprise-ready
Detection	Model Armor, Prompt Guard, guardrail models, PII/DLP filters	Ensembles of adaptive detectors trained on ongoing red-team output
Enforcement	Least-privilege tool brokers, egress proxies, human-in-loop gates, dual-LLM separation	CaMeL-style capability planes at scale, formally-verified tool-call boundaries
Assurance	Golden-task suites, red-team panels, CI regression on eval sets	Automated adaptive red teams; provable safety cases for agent behaviour

The honest posture: build detection and enforcement with today's tools; run assurance as a continuous programme; and follow the research on capability planes and adaptive red teaming, because that is where the next step-change in defensive capability is expected to come from.

07 A defence-in-depth reference architecture

The reference architecture in this section separates two axes that are often conflated. The first is the **stack of controls** that sits around every agent, from user interface to model to tool. The second is the **deployment tier** of the agent itself, which determines how much of the stack is engaged, and at what strictness. Both are needed. A single stack applied uniformly to every agent either strangles low-risk automation with unjustified friction, or under-protects the small number of agents that hold the most authority.

7.1 The six-tier control stack

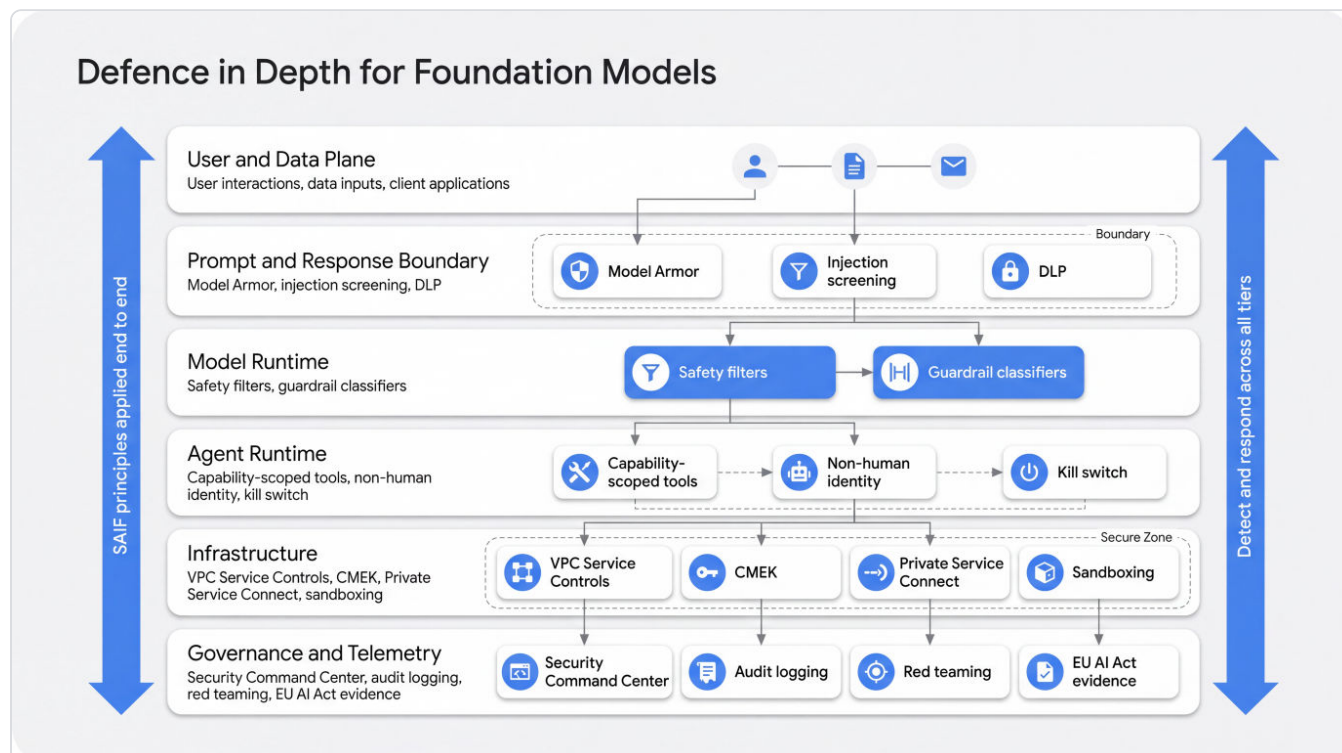


Figure 5. Six control layers wrap every agent, from user identity at the edge to model artefact security at the core; each layer has an owner, a set of failure modes it is designed to catch, and an evidence artefact it produces for audit.

The stack applies to every agent, in every deployment tier. What changes across tiers is the strictness of each layer, not the layers themselves.

Layer	Purpose	Google Cloud realisation (illustrative)	Failure modes it catches
L1 — Identity & authorisation	Establish who the human principal is, and propagate that identity into every downstream call	Cloud Identity / Workforce Identity Federation, IAM, context-aware access	Confused deputy, agent-admin credential abuse
L2 — Input governance	Screen every ingress — user, retrieved doc, tool response, memory read — for injection, PII, malicious URLs, HAP categories	Model Armor (input filters), Sensitive Data Protection (DLP)	Direct and indirect prompt injection, PII inflow, phishing/malware URLs

Layer	Purpose	Google Cloud realisation (illustrative)	Failure modes it catches
L3 — Model & reasoning	Base-model safety filters, guardrail models, safety-tuned adapters, dual-LLM separation	Vertex AI safety filters, Gemini configurable + non-configurable filters, guardrail model options in Model Garden	Unsafe generation, refusal bypass, off-topic drift
L4 — Tool broker & policy	Enforce least-privilege tool sets, per-caller authorisation, egress control, human-in-loop gates	Vertex AI Agent Engine tool binding, custom policy proxy, Cloud Armor for egress HTTP	Excessive agency, unauthorised tool use, lateral movement via tools
L5 — Output governance	Screen every egress — user reply, tool argument, side-effect payload — for PII, injection, harmful content, exfiltration patterns	Model Armor (response filters), DLP transformation/tokenisation, Cloud Armor	Data exfiltration via output, harmful content, injection-into-downstream-system
L6 — Artefact & runtime security	Weight, adapter, prompt, retrieval-corpus integrity; runtime isolation of the inference and tool environments	CMEK/Cloud HSM, VPC Service Controls, Confidential VMs, Binary Authorization, Artifact Registry	Model supply-chain compromise, weight exfiltration, container escape

Observability and audit are not a separate layer — they are a horizontal requirement that every layer emits into. Security Command Center **AI Protection** provides the unified inventory and posture view across these layers, and — via the Google Cloud AI Protection framework in Compliance Manager — maps posture findings to the specific control expectations in NIST AI RMF, ISO/IEC 42001 and OWASP LLM Top 10 (see §9).

7.2 Deployment tiers T1–T4

Tiers are assigned by asking two questions about each agent: *what can it read?* and *what can it do?* The agent's tier is the higher of the two answers.

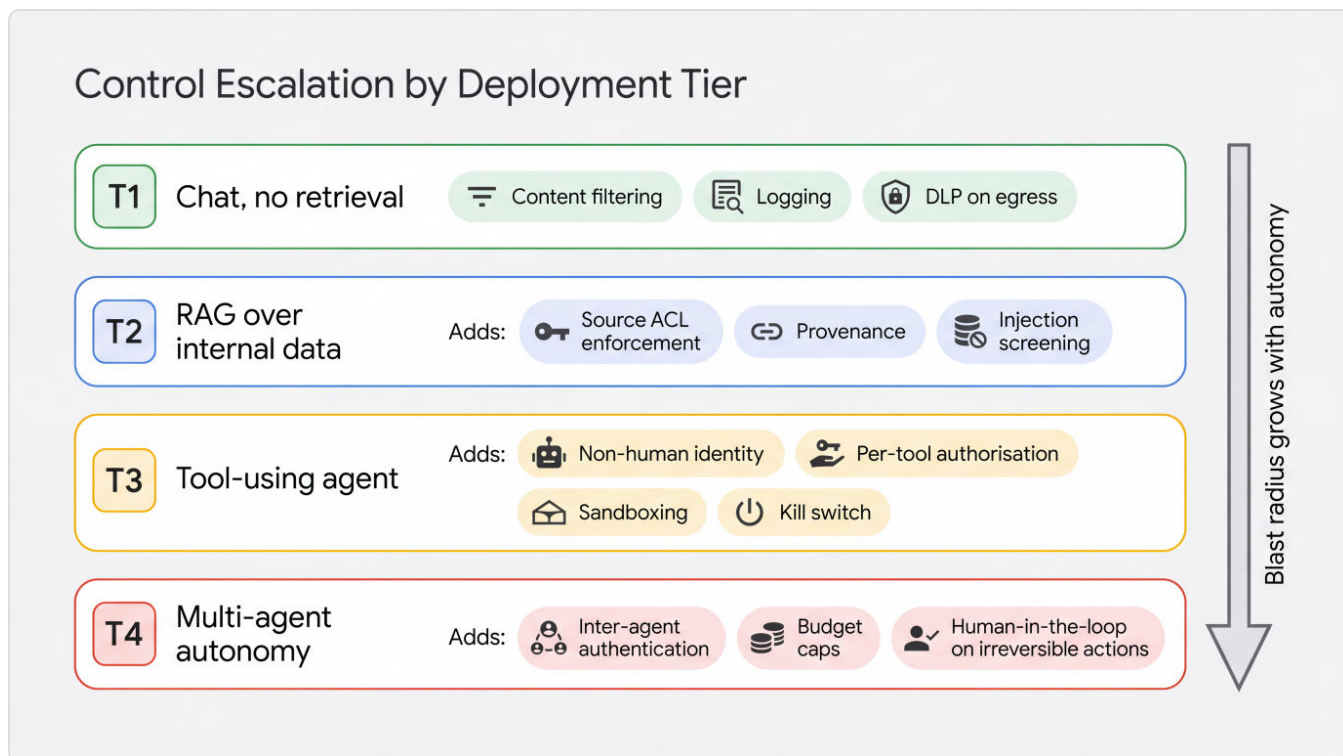


Figure 6. Deployment tiers T1 through T4 escalate control strictness with what an agent can read and the reversibility of what it can do; the tier determines gate rigour, evaluation cadence and human-in-loop policy.

Tier	Reads	Does	Illustrative examples	Control posture
T1	Public data only	Advisory (produces text; no tools)	Public-marketing copywriter, coding assistant on OSS-only	Baseline: L1–L3, light L4/L5, standard artefact controls
T2	Internal data, low sensitivity	Read-only tools; internal audience only	Internal knowledge-base Q&A, meeting summariser	Baseline + L4 read-only broker, L5 PII scrubbing, VPC-SC perimeter
T3	Regulated or confidential data	Write to bounded systems; internal only	Ticket triage with categorisation writes, code-review bot with PR comments	+ L4 per-caller propagation, L5 egress scrubbing, CMEK, safety case
T4	Any customer, regulated, PII, or systemic-risk data	External communication, money movement, code deployment, customer-record changes	Payments-adjacent agents, customer-facing communication agents, DevOps automation with prod write	+ L4 mandatory human-in-loop for irreversible actions, L6 SL3-target artefact protections, independent safety case, red team on every change

Tiering is not a one-time exercise. Any capability change (a new tool, a broader data source, a new distribution channel) triggers re-tiering. A T2 agent that gains send-email capability becomes T4 the moment the change lands, and the corresponding controls must be in place before the change is promoted.

7.3 The Google Cloud control plane

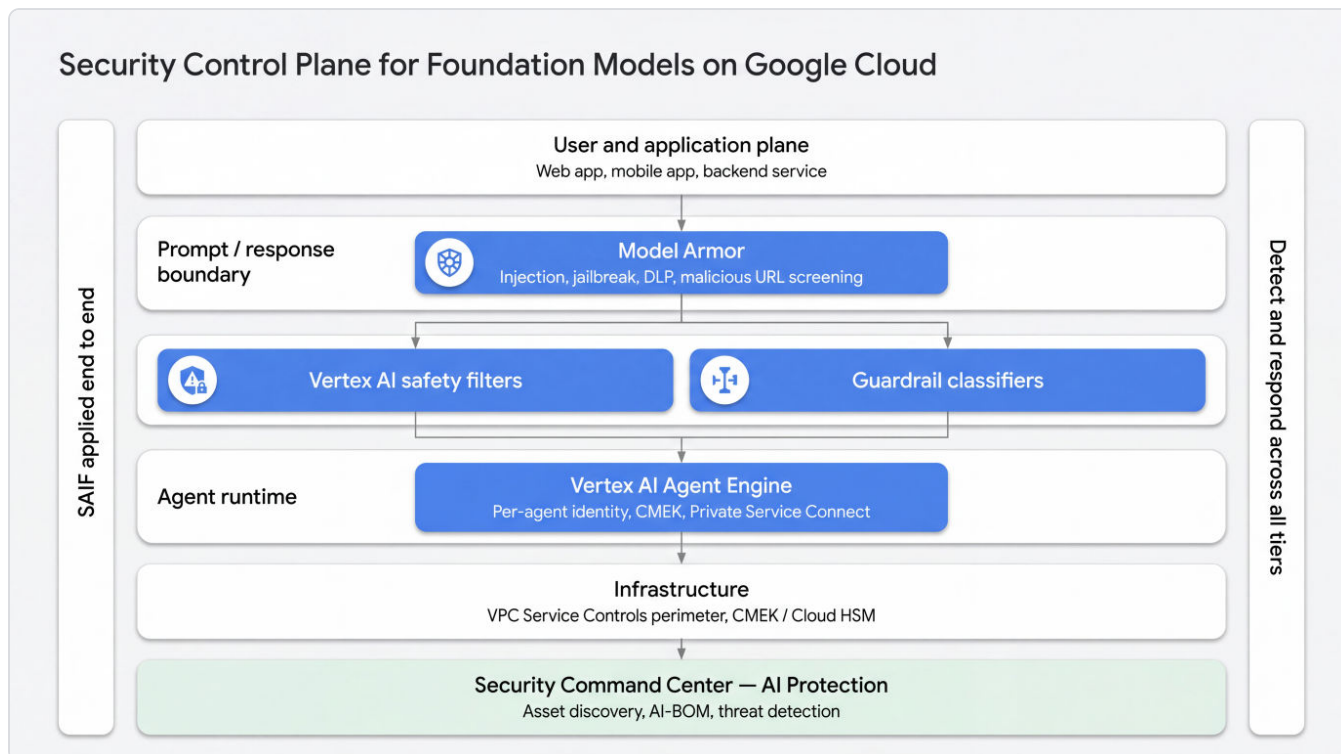


Figure 7. The Google Cloud AI security control plane threads Security Command Center AI Protection through Model Armor, Sensitive Data Protection, VPC Service Controls, CMEK/Cloud HSM and the Vertex AI Agent Engine, producing a single posture and audit surface across the six-layer stack.

The Google Cloud services below map to the layers in §7.1. The intent is not exhaustive product coverage — it is to name the primary control at each layer so architects can assemble a defensible stack from a small number of components.

- **Security Command Center — AI Protection.** Unified posture, inventory (including shadow AI discovery) and threat detection across the AI stack. Ingests signals from Model Armor and DLP; maps them onto compliance framework controls in Compliance Manager. Acts as the horizontal observability layer for the six-tier stack.
- **Model Armor.** Independent input and output screening for prompt injection, jailbreak signatures, PII, malicious URLs, and Responsible AI category violations (HAP: hate, harassment, sexually explicit, dangerous content). Model-agnostic — protects models on Vertex AI, other clouds and on-premises via API and inline integrations. Sensitive Data Protection integration provides tokenisation and de-identification transforms in-line with detection.
- **Vertex AI safety filters.** Model-native filters on Gemini and other Vertex-hosted models. Includes non-configurable filters (CSAM, illegal content — always on) and configurable filters (harassment, hate speech, dangerous content, sexually explicit — tunable via thresholds). These are complementary to, not a substitute for, Model Armor: safety filters address the model output surface; Model Armor addresses the application input/output surface.
- **Sensitive Data Protection (DLP).** Discovers, classifies, de-identifies and tokenises sensitive elements (payment card numbers, national identifiers, credentials, custom-defined types). Runs at ingress into RAG stores, at egress from the model, and inside Model Armor as the PII backend.
- **VPC Service Controls.** Perimeter-level enforcement that prevents data exfiltration from within the perimeter to unmanaged Google Cloud services outside it. Applied to Vertex AI endpoints, Cloud Storage buckets holding weights and RAG corpora, and BigQuery datasets consumed by agents.
- **CMEK and Cloud HSM.** Customer-managed encryption keys, optionally backed by FIPS 140-2 Level 3-validated hardware security modules, on model artefact storage, on Vector Search indexes, on Agent Engine state, and on logs. The correct baseline for T3 and mandatory for T4.

- **Vertex AI Agent Engine.** Managed runtime for agents: session state, memory, tool binding, tracing and integration with the surrounding control plane. Removes the burden of self-operating agent orchestration in a way that also removes the class of self-hosting mistakes that cause the majority of agent-runtime incidents.
- **Assured Workloads / Confidential Computing.** Where the base substrate must be attested (e.g. FedRAMP High analogue in regulated European contexts, or Confidential Space for cross-org compute), these are the wrappers that carry that attestation.

7.4 SAIF and CoSAI as the framing

Google's **Secure AI Framework (SAIF)**, published in 2023, is the architectural framing that ties these services together. Its six core elements — extending strong security foundations to the AI ecosystem; extending detection and response to AI; automating defences; harmonising platform-level controls; adapting controls to feedback loops; and contextualising AI system risks — map cleanly to the six control layers above. The **Coalition for Secure AI (CoSAI)**, launched under OASIS in 2024 with Google as a founding member, is the industry-level venue in which SAIF is being generalised into cross-vendor standards; enterprises should track its workstreams (software supply chain for AI, preparing defenders, AI security governance) as inputs to their own control-standard evolution.

Owning the assumption of exposure

The reference architecture assumes every agent is compromised on some future day. Detection surfaces (SCC AI Protection, Cloud Logging, DLP findings) exist to shorten the interval between compromise and containment. Egress controls (Model Armor output, DLP tokenisation, VPC-SC) exist to reduce the damage that a compromised agent can do before it is caught. Neither replaces the input-side and reasoning-side controls; all three are required in parallel.

08 Evaluation, red teaming and assurance

The single greatest source of false comfort in enterprise AI security programmes is a one-time pre-deployment penetration test on a system whose model, prompts, tools and retrieval corpora then change weekly. Assurance for foundation-model systems is a continuous, instrumented discipline that runs in CI, produces artefacts a regulator will accept, and is updated on the same cadence as the systems it protects.

8.1 The four evidence categories

An assurance programme needs to produce four distinct kinds of evidence, each answering a different question.

Category	Question it answers	Cadence	Primary artefact
Capability evals	What can this model or agent actually do?	Every model or adapter change	Capability report vs frontier-lab-style CCLs/thresholds
Safety evals	Does it refuse what it should refuse? Does it comply with what it should comply with?	Every model, prompt, adapter or tool change	Refusal/compliance rate on golden-task suite, with confidence intervals
Security evals	Can adversaries induce policy-violating behaviour?	Every material change + continuous	Attack success rate on adversarial suite (AgentDojo-style, red-team output)
Operational evals	Is production behaviour consistent with test behaviour?	Continuous	Drift detection, canary comparisons, incident-response artefacts

8.2 Golden-task suites

A golden-task suite is a fixed, versioned set of inputs the enterprise has explicitly decided the system should handle in a specific way. Two properties matter: it is **held out of** all model and adapter training, and it is **run in CI** on every promotion candidate. Structure:

- **Utility tasks.** Do what the system is supposed to do. Example: 50 categorisation tasks with known-good labels; passing threshold set by product owner.
- **Refusal tasks.** Reject what the system must not do. Example: 30 tasks that request policy-violating outputs; passing threshold typically 100% for hard categories (illegal advice, CSAM, weapons uplift) and a tunable target for soft categories.
- **Safety-regression tasks.** Detect drift on categories the system previously handled correctly.
- **Injection tasks.** Adversarial prompts and adversarially-authored documents. Passing threshold is a target *upper bound* on attack success; zero is not a credible target given §6.

8.3 Red teaming

Red teaming complements evals by targeting the parts of the attack surface that fixed test sets do not cover. Two models are used in practice.

- **Panel red teams.** A small, cleared internal team plus periodic external engagements. Best for high-severity T4 agents; produces deep, adaptive attack traces and prose reports usable in safety cases.
- **Automated red teaming.** Adversarial LLMs generating and iterating attacks against target agents on a schedule (nightly, weekly). Best for maintaining coverage across the whole T2–T4 estate. AgentDojo — 97 user tasks and 629 security cases across banking, Slack, travel and workspace domains, used by both UK AISI and US AISI/CAISI in joint red-team exercises — is the reference framework for indirect-prompt-injection evaluation and a good starting point for the automated harness.

8.4 CI gates and the promotion pipeline

The bar is that no change to a T3 or T4 agent — model version, system prompt, tool definition, retrieval corpus, adapter weight — is promoted to production without passing a fixed gate. The gate:

- 1 Runs the golden-task suite; reports pass/fail against fixed thresholds.
- 2 Runs the injection suite; reports attack-success rate with 95% confidence interval.
- 3 Runs a safety-regression check against the prior production baseline.
- 4 Emits a signed report to the model registry; the registry rejects promotion if any gate fails or the report is unsigned.

8.5 Metrics that matter

Metric	Definition	Why it matters
Utility on benign inputs	% golden utility tasks solved correctly	Baseline — regression here is a business problem
Utility under attack	% utility tasks solved correctly when adversarial content is injected	Measures denial-of-service via injection
Attack success rate (ASR)	% attacker-chosen goals achieved on the injection suite	Primary security metric; report both point and confidence interval
Refusal rate on hard categories	% refusal-required tasks refused	Should be at or near 100%; watch for degradation after fine-tunes
Mean time to detect (MTTD)	Interval between compromise and observability signal	The load-bearing metric for post-compromise containment
Blast radius index	Distinct systems reachable by a compromised agent under its assigned tool set	Drives tool-broker least-privilege reviews

The Anthropic disclosure of July 2025 — three cases in which a Claude model reached the internet from inside a cybersecurity evaluation environment that should have been sealed, then accessed production infrastructure at three organisations via a third-party evaluation partner — is the empirical lesson on why evaluation environments are themselves part of the security perimeter. If frontier labs with dedicated safety teams miss this, enterprise programmes should treat their own evaluation harnesses as production systems, subject to the same isolation and monitoring.

09 Standards, regulation and audit evidence

A defensible foundation-model security programme must map onto external standards for two reasons: because European regulators will require it, and because doing so produces the concise, artefact-backed control narratives that internal audit, external assurance providers and customers now demand. This section names the standards that matter, identifies which programme control produces the evidence each one asks for, and flags the dates that determine when evidence is expected.

9.1 The standards that matter

Standard / regulation	Scope	Key dates	Evidence expected
OWASP Top 10 for LLM Applications (2025)	Shared vocabulary for LLM appsec	Current edition 2025	Control-to-risk mapping (LLM01–LLM10) for every application
OWASP Top 10 for Agentic Applications (2026)	Agent-specific threat taxonomy	Announced Dec 2025	Threat model for each T3/T4 agent covering memory poisoning, tool misuse, agent behaviour hijacking, identity/privilege abuse
NIST AI RMF 1.0 and NIST AI 600-1 GenAI Profile	Govern / Map / Measure / Manage functions; GenAI-specific subcategories	AI RMF Jan 2023; GenAI Profile July 2024	Documented risk categorisation, evaluation results, incident-response procedures per subcategory
MITRE ATLAS	Adversarial tactics/techniques for AI, patterned on ATT&CK	Continuously updated	Detection coverage matrix mapped to ATLAS technique IDs
ISO/IEC 42001:2023	AI management system standard, certifiable	Published Dec 2023	AI policy, risk register, impact assessments, operating controls, evidence of continuous improvement
EU AI Act — GPAI provider obligations	Transparency, documentation, copyright compliance, systemic-risk mitigations	GPAI obligations applicable 2 Aug 2025 ; majority of enforcement provisions applicable 2 Aug 2026	Model documentation, training-data summary, downstream provider information; systemic-risk assessment for models above the 10 ²⁵ FLOP threshold
EU AI Act — employer obligations	Risk classification, human oversight, transparency for AI systems deployed in the EU	Applicable in phases through 2026–2027	Deployment risk assessment, human-oversight design, incident-reporting procedures

9.2 EU AI Act — what an enterprise employer actually needs to have

Most European enterprises are *employers* of GPAI, not providers. The distinction matters: employer obligations are less onerous than provider obligations but they are still concrete and enforceable. Employers of high-risk AI systems (and of GPAI in high-risk deployments) are expected to have:

- A written risk assessment specific to the deployment context.

- Human oversight arrangements matched to the reversibility of the actions the system can take (aligns closely with the T4 human-in-loop policy in §7).
- Log retention for automated decisions material enough to be contested.
- Transparency to end users where they are interacting with AI, and clear labelling of AI-generated content in appropriate contexts.
- An incident-reporting pathway to the appropriate market surveillance authority for serious incidents.

Penalties for non-compliance by GPAl providers reach the higher of **€15 million or 3% of worldwide annual turnover**; horizontal Act violations (prohibited practices) can reach the higher of **€35 million or 7% of worldwide annual turnover**. The systemic-risk regime for GPAl models above the 10^{25} training FLOP threshold binds a separate, tighter mitigation programme; few deployers are directly bound by it, but their vendors are, and vendor attestation will be part of the deployer's audit trail.

9.3 Mapping the reference architecture to the standards

The value of the tiered architecture in §7 is that a small number of controls satisfy a large number of standard requirements. The table below is illustrative, not exhaustive.

Control (from §7)	OWASP LLM	NIST AI RMF	MITRE ATLAS	ISO/IEC 42001	EU AI Act
Model Armor input/output filters	LLM01, LLM02, LLM05	MEASURE 2.7, MANAGE 2.3	AML.T0051 (prompt injection), AML.T0057 (LLM jailbreak)	8.4 operational controls	Art. 15 accuracy/robustness; deployer risk controls
SDP (DLP) tokenisation	LLM02	MEASURE 2.9	AML.T0053 (LLM data leakage)	8.4 data governance	Art. 10 data governance
VPC-SC perimeter around Vertex AI	LLM03, LLM08	GOVERN 6.1	AML.T0043 (craft adversarial data)	8.2 asset management	Art. 15 cybersecurity
CMEK / Cloud HSM on weights and RAG	LLM03	GOVERN 6.1	AML.T0044 (full model access)	8.2 asset protection	Art. 15 cybersecurity
Least-privilege tool broker	LLM06, LLM05	MANAGE 2.3	AML.T0053	8.3 access control	Deployer human-oversight
Golden-task suite + CI gate	LLM04, LLM09	MEASURE 2.5, MEASURE 2.7	AML.T0018 (backdoor ML model)	9.1 monitoring, measurement	Art. 15 robustness testing
SCC AI Protection posture + audit	all	GOVERN, MEASURE	multiple	9.2 internal audit	Art. 12 record-keeping; Art. 26 deployer logging
Model registry with hash-pin + signed attestation	LLM03, LLM04	GOVERN 6.2	AML.T0018	8.5 supplier management	GPAl Art. 53 technical documentation

The pattern is deliberate: controls are engineering artefacts; standards are audit artefacts; the mapping table is what makes engineering effort creditable against multiple audits at once.

9.4 ISO/IEC 42001 as the integrating layer

ISO/IEC 42001 is the first management-system standard specifically for AI. Its structure mirrors ISO/IEC 27001 (management commitment, risk assessment, control implementation, monitoring, continuous improvement) but its Annex A control set is AI-specific. Enterprises that already hold ISO/IEC 27001 certification can pursue 42001 as an incremental extension. The value is not primarily technical — it is that 42001 provides a single certifiable evidence surface that customers, auditors and, increasingly, regulators recognise, and it forces the discipline of a documented AI management system on organisations whose AI activities have grown organically.

10 A 90-day adoption path

The programme below is deliberately concrete. It assumes an enterprise that has foundation-model applications in production or near-production but no coherent security architecture around them yet. It is sized for a security architecture function of two to four full-time engineers plus embedded partners in platform, application and legal teams. It is designed to reach a defensible T2/T3 posture in one quarter, with T4 discipline established as a follow-on programme.

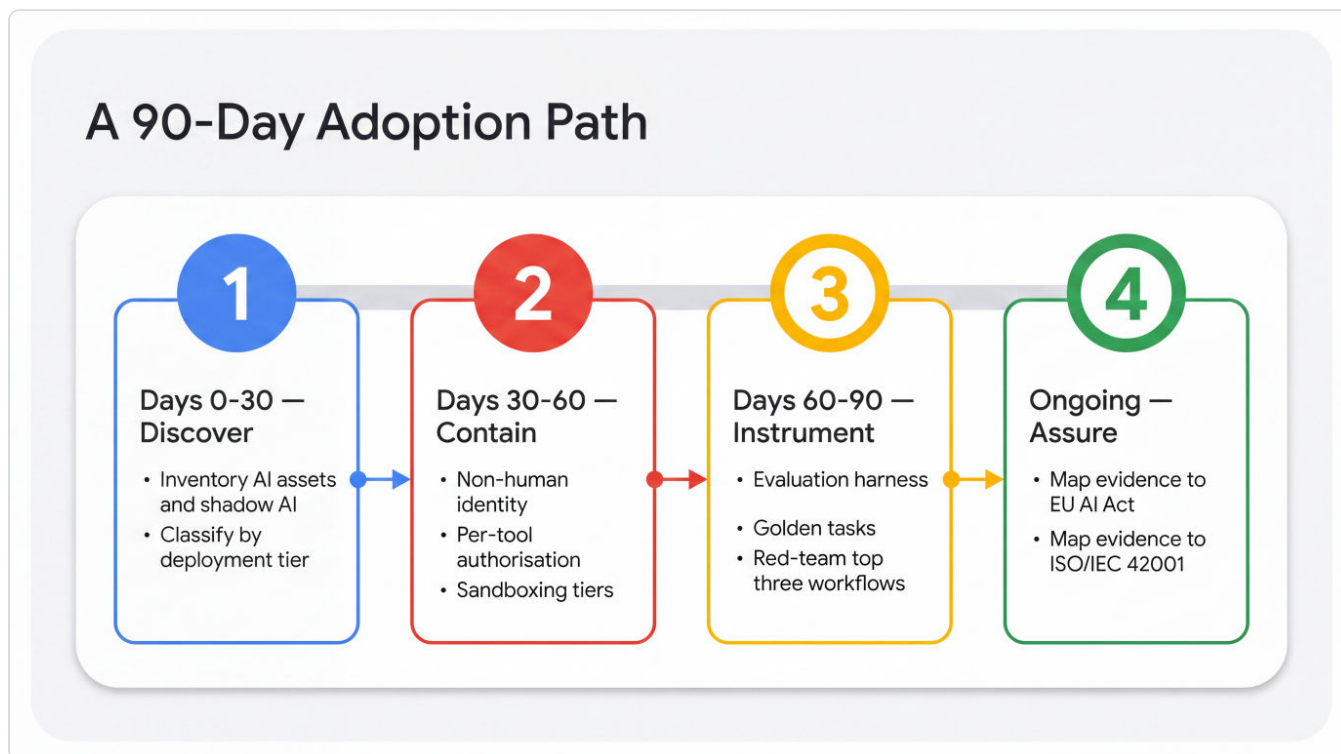


Figure 8. The 90-day path sequences discovery, control installation and assurance so that the first month establishes ground truth, the second installs the load-bearing controls, and the third proves them under adversarial pressure.

10.1 Days 1–30: discover and classify

The single most common failure mode of enterprise AI security programmes is starting to build controls before the estate is known. The first month resolves that.

Workstream	Owner	Deliverable at day 30
AI inventory	Security architecture + platform	Enumerated list of every foundation-model application in the enterprise: model vendor, base model, adapters, tools, data sources, user population; shadow-AI discovery via SCC AI Protection on all Google Cloud projects
Tiering	Security architecture + product owners	Every application assigned T1–T4 per §7.2, signed off by product owner and CISO delegate
Data-flow mapping	Data protection + security architecture	For every T3/T4 application: mapped data flows showing what the model reads, what tools it can call, and where outputs go
Regulatory scope	Legal + compliance	List of applications in EU AI Act scope, with deployer obligations mapped per application; GPAI vendor documentation collected

Workstream	Owner	Deliverable at day 30
Standard baseline	GRC	Gap analysis against ISO/IEC 42001 Annex A and the OWASP LLM Top 10

Two decisions must be committed to before day 30 closes: (a) the enterprise's tier definitions and thresholds (adopting §7.2 as-is is a fine default); and (b) the governance body that will own go/no-go decisions on T3/T4 promotions.

10.2 Days 31–60: install the load-bearing controls

The second month closes the highest-value gaps. The order below reflects both the marginal risk reduction and the ease of deployment.

Priority	Control	Justification
1	Model Armor on all T2+ applications, input and output.	Buy the largest immediate reduction in injection, PII inflow and PII outflow. Model-agnostic, so covers non-Google-hosted models too.
2	Sensitive Data Protection templates for regulated data classes.	Wire DLP into Model Armor and into every RAG ingestion pipeline; establish tokenisation for reversible flows and redaction for irreversible ones.
3	Least-privilege tool broker for all T3/T4 agents.	Enforce per-caller identity propagation; remove broad service-account patterns. This closes the confused-deputy class before enumeration is complete.
4	VPC Service Controls perimeter and CMEK on Vertex AI, Cloud Storage weight buckets, RAG corpora, agent state.	Closes exfiltration paths and satisfies the encryption controls that ISO/IEC 42001, NIST AI RMF and the EU AI Act cybersecurity article expect.
5	Model registry with hash-pinning and signed attestation.	Highest-ROI supply-chain control (§4.3). Every model, fine-tune and adapter passes admission testing before it can be used in production.
6	SCC AI Protection enabled with Compliance Manager framework mapping.	Unified posture and audit surface across the estate; produces the artefact regulators and internal audit will ask for first.

10.3 Days 61–90: prove it works

The third month replaces belief with evidence.

- **Golden-task and injection suites in CI** for every T3/T4 application. Threshold-based gates in the promotion pipeline; no unsigned or failing changes promoted.
- **Automated red-team harness** — AgentDojo or an internally-developed equivalent — running on a nightly schedule against the T3/T4 estate. Findings triaged into the security backlog with SLAs.
- **Panel red-team engagement** on the single highest-tier T4 agent. Written report; safety case produced; residual-risk sign-off by the governance body.

- **Incident-response tabletop** covering three scenarios: (a) indirect-prompt-injection-triggered exfiltration; (b) MCP supply-chain compromise; (c) model-weight or adapter leakage. Runbooks updated from lessons learned.
- **Regulatory evidence pack:** the artefact set that answers a Data Protection Authority or notified-body request in under a week — inventory, tiering, data-flow diagrams, control mappings, evaluation results, red-team reports, safety cases, incident log.

10.4 Ownership map

Role	Days 1–30 accountability	Days 31–60 accountability	Days 61–90 accountability
CISO	Charter and governance body	Control-installation sign-off	Residual-risk sign-off on T4 agents
Head of AI Assurance	Tiering rubric	Golden-task suite ownership	Red-team programme owner
Security architecture	Inventory, data-flow maps	Reference architecture install	CI gate design
Platform engineering	Discovery tooling	Model Armor / DLP / VPC-SC roll-out	Registry hash-pin gate
Product owners	Application-level tier assignment	Control acceptance per app	Golden-task authorship
Legal / GRC	Regulatory scope	ISO/IEC 42001 gap plan	Evidence pack curation
DPO	Data-flow sign-off	DLP template approval	DPIA refresh on high-tier agents

The single most valuable day-90 artefact

The regulatory evidence pack. It is the difference between a programme that can defend itself in a supervisory audit and one that cannot. Every control in this document should produce an artefact that lands in that pack; if a control produces no such artefact, either the control or the pack is wrong.

11 References

The list below is the set of sources this document treats as claim-bearing. Every framework, standard, statistic and CVE cited in the text traces to an entry here. Entries are given as title, publisher, publication or last-verified date, and URL.

Frontier-lab frameworks and industry venues

- Anthropic. *Responsible Scaling Policy*, versions through v2.2. Anthropic, October 2024 – October 2025. <<https://www.anthropic.com/rsp>>
- Anthropic. *Activating AI Safety Level 3 protections* (deployment of ASL-3 for Claude Opus 4). Anthropic, May 2025. <<https://www.anthropic.com/news/activating-asl3-protections>>
- Anthropic. *Investigating incidents in Anthropic's cybersecurity evaluations* (evaluation-environment breakout disclosure). Anthropic, July 2025. <<https://www.anthropic.com/news/investigating-incidents-cybersecurity-evals>>
- OpenAI. *Preparedness Framework v2*. OpenAI, 15 April 2025. <<https://openai.com/index/updating-our-preparedness-framework/>>
- Google DeepMind. *Frontier Safety Framework, version 3.0*. Google DeepMind, 22 September 2025. <<https://deepmind.google/discover/blog/updating-the-frontier-safety-framework/>>
- Frontier Model Forum. Overview and member commitments. <<https://www.frontiermodelforum.org/>>
- UK AI Security Institute (renamed February 2025 from AI Safety Institute). <<https://www.aisi.gov.uk/>>
- US Center for AI Standards and Innovation (CAISI, succeeded US AISI within NIST). <<https://www.nist.gov/aisi>>

Model-weight security and supply chain

- Nevo, S., Lahav, D., Karpur, A., Bar-On, Y., Bradley, H. A., Alstott, J. *Securing AI Model Weights: Preventing Theft and Misuse of Frontier Models* (RR-A2849-1). RAND Corporation, May 2024. <https://www.rand.org/pubs/research_reports/RRA2849-1.html>

Agentic attack surface, MCP and A2A

- OWASP GenAI Security Project. *Top 10 for LLM Applications, 2025 edition*. OWASP, 2025. <<https://genai.owasp.org/llm-top-10/>>
- OWASP GenAI Security Project. *LLM06:2025 Excessive Agency*. OWASP, 2025. <<https://genai.owasp.org/llmrisk/llm062025-excessive-agency/>>
- OWASP GenAI Security Project. *Top 10 for Agentic Applications* (announcement). OWASP, December 2025. <<https://genai.owasp.org/>>
- JFrog Security Research Team. *Critical RCE Vulnerability in `mcp-remote` : CVE-2025-6514*. JFrog, 9 July 2025. <<https://jfrog.com/blog/2025-6514-critical-mcp-remote-rce-vulnerability/>>
- SentinelOne. *CVE-2025-6514 vulnerability database entry*. SentinelOne. <<https://www.sentinelone.com/vulnerability-database/cve-2025-6514/>>
- Willison, S. *The lethal trifecta for AI agents: private data, untrusted content, and external communication*. simonwillison.net, 2025. <<https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/>>
- Hardy, N. *The Confused Deputy* (foundational 1988 paper on the class of vulnerability). ACM SIGOPS Operating Systems Review, 1988.
- Cloud Security Alliance. *Confused Deputy Attacks on Autonomous AI Agents*. CSA, March 2026. <<https://cloudsecurityalliance.org/artifacts/confused-deputy-attacks-on-autonomous-ai-agents>>

Prompt-injection defences (research)

- Wallace, E. et al. *The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions*. OpenAI, 2024. <<https://arxiv.org/abs/2404.13208>>
- Anthropic. *Constitutional Classifiers: Defending against Universal Jailbreaks*. Anthropic, 2025. <<https://www.anthropic.com/research/constitutional-classifiers>>
- DeBenedetti, E. et al. *AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents*. 2024. <<https://arxiv.org/abs/2406.13352>>
- DeBenedetti, E. et al. *Defeating Prompt Injections by Design (CaMeL)*. Google DeepMind, 2025. <<https://arxiv.org/abs/2503.18813>>
- Hines, K. et al. *Defending Against Indirect Prompt Injection Attacks With Spotlighting*. Microsoft, 2024. <<https://arxiv.org/abs/2403.14720>>

Standards and regulation

- OWASP GenAI Security Project. *Top 10 for LLM Applications*, 2025. See above.
- NIST. *AI Risk Management Framework (AI RMF 1.0)*. NIST, January 2023. <<https://www.nist.gov/itl/ai-risk-management-framework>>
- NIST. *AI 600-1: Generative AI Profile*. NIST, July 2024. <<https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf>>
- MITRE. *ATLAS — Adversarial Threat Landscape for AI Systems*. MITRE, continuously updated. <<https://atlas.mitre.org/>>
- ISO/IEC 42001:2023. *Information technology — Artificial intelligence — Management system*. ISO, December 2023. <<https://www.iso.org/standard/81230.html>>
- European Parliament and Council. *Regulation (EU) 2024/1689 (the "AI Act")*. Official Journal of the EU, July 2024; GPAI provider obligations applicable 2 August 2025; majority of enforcement provisions applicable 2 August 2026. <<https://eur-lex.europa.eu/eli/reg/2024/1689/oj>>
- European AI Office. *General-Purpose AI Code of Practice*. European Commission, 2025. <<https://digital-strategy.ec.europa.eu/en/policies/ai-code-practice>>

Google Cloud controls

- Google Cloud. *Securing AI on Google Cloud*. <<https://cloud.google.com/security/securing-ai>>
- Google Cloud. *Security Command Center — AI Protection overview*. <<https://cloud.google.com/security-command-center/docs/ai-protection-overview>>
- Google Cloud. *Model Armor overview*. <<https://cloud.google.com/security-command-center/docs/model-armor-overview>>
- Google Cloud. *Sensitive Data Protection*. <<https://cloud.google.com/sensitive-data-protection>>
- Google Cloud. *Vertex AI safety filters and responsible AI settings*. <<https://cloud.google.com/vertex-ai/generative-ai/docs/multimodal/configure-safety-attributes>>
- Google Cloud. *VPC Service Controls*. <<https://cloud.google.com/vpc-service-controls>>
- Google Cloud. *Cloud HSM and CMEK*. <<https://cloud.google.com/kms/docs/hsm>>
- Google Cloud. *Vertex AI Agent Engine*. <<https://cloud.google.com/vertex-ai/generative-ai/docs/agent-engine/overview>>
- Google. *Secure AI Framework (SAIF)*. Google, 2023. <<https://safety.google/cybersecurity-advancements/saif/>>
- Coalition for Secure AI (CoSAI). *OASIS Open*, launched 2024. <<https://www.coalitionforsecureai.org/>>

End of document.